The Residual Stream Does Not Recover: Bijective Token Permutations Create Persistent Distortions in Pretrained Transformers

Ari Holtzman and NeuriCo

Abstract

We investigate whether a pretrained transformer's residual stream can adapt to a bijective token permutation—a systematic shuffling of which token IDs map to which surface forms—applied without any in-context demonstrations. Across experiments on GPT-2 SMALL and PYTHIA-410M, we find that the model does *not* recover "modulo the shuffle." While cosine similarity between clean and ciphered hidden states increases at later layers, we show this is largely an architectural artifact: LAYERNORM drives all inputs—including random tokens—toward similar representations, with inter-text cosine similarity exceeding 0.96 in GPT-2 SMALL. A permutation-adjustment matrix designed to undo the cipher in embedding space makes similarity *worse*, confirming the cipher's effect propagates non-linearly. The logit lens further shows the model never predicts the cipher-consistent next token at any layer, with mean ranks above 5,900 out of 50,257. PYTHIA-410M, with weaker architectural convergence, reveals that ciphered representations sit 30% above random but 30% below clean at the final layer—partial structural extraction without genuine decoding. Our results demonstrate that the embedding lookup creates a persistent, non-invertible distortion: the forward pass cannot undo what amounts to a random permutation of 50,000 semantic embeddings.

1 Introduction

Pretrained language models encode rich semantic knowledge in their embedding layers and residual streams. A natural question arises: if we systematically remap *which* token IDs correspond to *which* surface forms—applying a bijective permutation $f : \mathcal{V} \to \mathcal{V}$ to the input—can the model's internal representations adapt? After processing enough permuted text, does the residual stream converge to something resembling its normal state, "modulo the shuffle"?

This question matters for several reasons. First, it probes how deeply a transformer's "understanding" is tied to specific token identities versus distributional structure. Second, it has practical implications for adversarial robustness: if a simple bijective shuffle can destroy a model's predictions even though the statistical structure of the input is preserved, this reveals a fundamental rigidity in how transformers process language. Third, it informs the growing literature on cipher-based jailbreaks [Yuan et al., 2023], where attackers encode harmful prompts via substitution ciphers to bypass safety filters.

What is missing in existing work? Recent work by Fang et al. [2025] demonstrates that models *can* learn bijective ciphers from in-context demonstrations, with the logit lens showing progressive cipher decoding at deeper layers. However, their analysis focuses on the *with-demonstrations* setting and measures only top-token predictions, not the full geometry of the residual stream. No prior work has systematically measured how the residual stream's *structure*—its direction, magnitude, and representational content—changes under bijective token permutation in the zero-shot setting, or whether there exists a linear transformation that maps ciphered hidden states back to clean ones.

We address this gap directly. We extract residual stream activations at every layer of GPT-2 SMALL (12 layers) and PYTHIA-410M (24 layers) under three conditions: clean text, bijectively ciphered text, and random token replacement. We compare these using cosine similarity, a permutation-adjustment matrix, the logit lens, and centered kernel alignment (CKA). Our key innovation is the *permutation-adjusted similarity* test: if the residual stream is truly "normal modulo the shuffle," then transforming ciphered hidden states by the cipher's inverse in embedding space should dramatically increase similarity to clean states.

Our results are clear: the model does not recover. The permutation adjustment makes similarity *worse*, not better. The high cosine similarity observed at later layers in GPT-2 SMALL (0.873 for full shuffle) is an architectural artifact—LAYERNORM drives *all* inputs, including random tokens (0.858), toward similar representations. The logit lens confirms the model never predicts cipher-consistent next tokens at any layer. In PYTHIA-410M, which exhibits weaker architectural convergence, ciphered representations sit between clean and random, suggesting partial distributional extraction without genuine cipher decoding.

We make the following contributions:

- We conduct the first systematic analysis of residual stream geometry under bijective token permutation, testing whether pretrained transformers can internally remap a cipher without demonstrations.
- We introduce a permutation-adjustment test that directly evaluates the "normal modulo the shuffle" hypothesis, finding it decisively refuted.
- We identify LAYERNORM-driven architectural convergence as a confound that inflates cosine similarity between *any* two inputs, cautioning against using raw cosine similarity as a measure of representational understanding.
- We validate our findings across two model architectures (GPT-2 SMALL and PYTHIA-410M), four shuffle rates, and five complementary analysis methods.

2 Related Work

Residual stream analysis and mechanistic interpretability. The residual stream—the sum of all previous layer outputs—serves as the primary information conduit in transformers. The *logit lens* [nostalgebraist, 2020] and its refinement, the *tuned lens* [Belrose et al., 2023], decode intermediate hidden states into vocabulary distributions, revealing how predictions evolve layer by layer. Lawson et al. [2025] extend this with multi-layer sparse autoencoders (MLSAEs) that decompose the residual stream into interpretable features across all layers simultaneously. Circuit discovery methods such as ACDC [Conmy et al., 2023] and edge attribution patching [Syed et al., 2023] identify which components drive specific behaviors. Zhao et al. [2024] show that knowledge conflicts between parametric and contextual information create detectable patterns in the residual stream. Nanda et al. [2023] demonstrate that transformers learn linear world-model representations in the residual stream. Our work extends this line of research by studying how the residual stream responds to systematic input perturbations rather than natural language variation.

Token permutation and cipher robustness. Sinha et al. [2021] show that masked language models pre-trained on randomly shuffled text retain 95–98% of downstream performance on most tasks, demonstrating that distributional co-occurrence statistics—not word order—drive much of pre-training success. Ravishankar et al. [2022] challenge this finding by showing shuffled-text models still encode some positional information through architectural inductive biases. Haviv et al. [2022] further demonstrate that transformers without explicit positional encodings learn position from causal attention masks alone. These results suggest that transformers have inherent structural biases that persist regardless of input ordering, which is relevant to understanding how the residual stream processes bijectively permuted tokens.

On the cipher side, Yuan et al. [2023] show that GPT-4 can interpret prompts encoded with Caesar ciphers, Morse code, and ASCII encoding—even without explicit demonstrations—indicating that large models develop latent cipher-processing capabilities during pre-training. Li et al. [2025] provide a systematic benchmark finding that LLMs struggle with complex ciphers but handle simple substitutions. Lee et al. [2024] show that training language models with explicit cipher/decipherment objectives improves multilingual transfer, suggesting that cipher-awareness can be learned.

In-context learning of bijective mappings. Most directly relevant to our work, Fang et al. [2025] introduce a framework using bijective substitution ciphers to disentangle task learning from task retrieval in in-context learning (ICL). They show that bijective ciphers consistently outperform non-bijective baselines across models and tasks (e.g., +4.8% on SST-2 for Llama-3.1-8B), with the gap widening as more demonstrations are provided. Crucially, their logit lens analysis shows that deeper layers progressively assign higher probability to the substituted token over the original, providing evidence that the residual stream can encode cipher mappings—but only when demonstrations are available.

Unlike Fang et al. [2025], who study the *with-demonstrations* setting and focus on behavioral metrics, we study the *zero-shot* setting and analyze the full geometry of the residual stream. We show that without demonstrations, the model fails to recover, and that apparent representational similarity at later layers is an architectural artifact rather than genuine cipher decoding.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{M}$ be a pretrained autoregressive transformer with vocabulary $\mathcal{V}$ of size $|\mathcal{V}|$ and embedding matrix $\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$, where d is the model dimension. We define a bijective token permutation $f : \mathcal{V} \to \mathcal{V}$ such that each token ID is mapped to exactly one other token ID. Given an input sequence $\boldsymbol{x} = (x_1, \ldots, x_n)$ with $x_i \in \mathcal{V}$, the ciphered sequence is $f(\boldsymbol{x}) = (f(x_1), \ldots, f(x_n))$.

Let $\boldsymbol{h}_\ell(\boldsymbol{x}) \in \mathbb{R}^{n \times d}$ denote the residual stream activations at layer ℓ when the model processes input $\boldsymbol{x}$. Our central question is whether there exists a transformation $\boldsymbol{T}$ (related to f) such that:

$$\boldsymbol{T} \cdot \boldsymbol{h}_\ell(f(\boldsymbol{x})) \approx \boldsymbol{h}_\ell(\boldsymbol{x}) \quad \text{for large } \ell. \tag{1}$$

If equation 1 holds, the residual stream is "normal modulo the shuffle"—the cipher's effect can be undone by a linear transformation in hidden space.

3.2 Bijective Cipher Construction

We construct f as a random bijective permutation with Zipfian frequency matching. Rather than permuting tokens uniformly at random, we partition $\mathcal{V}$ into logarithmic frequency bins based on each token's unigram frequency in a reference corpus and permute only within bins. This ensures that high-frequency tokens map to other high-frequency tokens, preserving the distributional statistics of the ciphered text.

We vary the *shuffle rate* $r \in \{0.3, 0.5, 0.7, 1.0\}$, which controls the fraction of the vocabulary that is permuted. At $r = 0.3$, 30% of tokens are shuffled while 70% retain their original identity; at $r = 1.0$, the entire vocabulary is permuted. All experiments use random seed 42. Bijectivity is verified by asserting that $f^{-1} \circ f$ equals the identity.

3.3 Models and Data

We use two pretrained autoregressive transformers accessed via TransformerLens [Nanda and Bloom, 2022]:

- GPT-2 SMALL: 12 layers, $d = 768$, $|\mathcal{V}| = 50{,}257$. Well-understood architecture with strong LAYERNORM effects.
- PYTHIA-410M: 24 layers, $d = 1024$, $|\mathcal{V}| = 50{,}304$. Provides cross-architecture validation with weaker LAYERNORM convergence.

Our dataset consists of 20–30 diverse English text samples of 2–4 sentences each, covering science, economics, literature, technology, and daily life. Texts are selected to span a range of vocabulary and syntactic structures. All experiments run on a single NVIDIA RTX A6000 (49 GB).

Table 1: Cosine similarity between clean and ciphered residual stream activations in GPT-2 SMALL across layers and shuffle rates. "Adjusted" applies the permutation-adjustment matrix $\boldsymbol{T}$ to ciphered states before comparison (full shuffle only). The random baseline uses non-bijective token replacement. Higher similarity at later layers is largely explained by architectural convergence (cf. table 2).

Condition	r	Embedding	Layer 6	Layer 12
Raw (clean vs. cipher)	0.3	0.927	0.849	0.942
Raw (clean vs. cipher)	0.5	0.873	0.756	0.912
Raw (clean vs. cipher)	0.7	0.822	0.687	0.896
Raw (clean vs. cipher)	1.0	0.743	0.579	0.873
Adjusted	1.0	0.256	0.068	0.417
Random	—	0.637	0.526	0.858

3.4 Analysis Methods

We compare residual stream activations across three conditions: (i) **clean**—original text, (ii) **cipher**—bijectively permuted text at various shuffle rates, and (iii) **random**—non-bijective random token replacement (lower bound).

Cosine similarity. We compute the cosine similarity between position-averaged hidden states $\bar{\boldsymbol{h}}_\ell^{\text{clean}}$ and $\bar{\boldsymbol{h}}_\ell^{\text{cipher}}$ at each layer ℓ:

$$\cos(\bar{\boldsymbol{h}}_\ell^{\text{clean}}, \bar{\boldsymbol{h}}_\ell^{\text{cipher}}) = \frac{\bar{\boldsymbol{h}}_\ell^{\text{clean}} \cdot \bar{\boldsymbol{h}}_\ell^{\text{cipher}}}{\|\bar{\boldsymbol{h}}_\ell^{\text{clean}}\| \, \|\bar{\boldsymbol{h}}_\ell^{\text{cipher}}\|}. \tag{2}$$

Permutation-adjusted similarity. To test equation 1, we compute a transformation matrix $\boldsymbol{T}$ as the least-squares solution to $\boldsymbol{E}_{\text{perm}} \cdot \boldsymbol{T} \approx \boldsymbol{E}$, where $\boldsymbol{E}_{\text{perm}}$ is the embedding matrix with rows permuted by f. We then measure $\cos(\boldsymbol{T} \cdot \bar{\boldsymbol{h}}_\ell^{\text{cipher}}, \bar{\boldsymbol{h}}_\ell^{\text{clean}})$. If the cipher's effect is linear and invertible, this should substantially exceed the raw similarity.

Logit lens. Following nostalgebraist [2020], we project each layer's hidden states through the model's unembedding matrix to obtain vocabulary distributions. We track the mean rank of the "correct" next token under both the cipher mapping and the original mapping.

Inter-text similarity control. To control for architectural convergence, we compute cosine similarity between position-averaged hidden states of *different clean texts*. If this baseline is already high, then high clean-vs-cipher similarity may reflect architectural properties rather than cipher decoding.

Centered kernel alignment (CKA). For PYTHIA-410M, we additionally compute CKA [Kornblith et al., 2019] between clean and ciphered representation matrices, which is robust to rotation and isotropic scaling.

Attention entropy. We measure the Shannon entropy of attention distributions across heads and positions to assess whether the cipher disrupts the model's attention patterns.

4 Results

4.1 Cosine Similarity Across Layers (GPT-2 SMALL)

Table 1 presents cosine similarity between clean and ciphered hidden states across layers in GPT-2 SMALL for all shuffle rates, alongside the random baseline and the permutation-adjusted condition.

Two patterns stand out. First, raw cosine similarity *increases* from the middle layers to the final layer across all shuffle rates—at first glance suggesting recovery. Second, the random baseline also reaches 0.858 at the final layer, nearly matching the full-shuffle cipher (0.873). The permutation-adjusted similarity is dramatically *lower* than raw similarity at every layer, reaching only 0.068 at layer 6. This directly refutes the hypothesis that the residual stream is "normal modulo the shuffle"—undoing the cipher in embedding space introduces noise rather than recovering the clean representation.

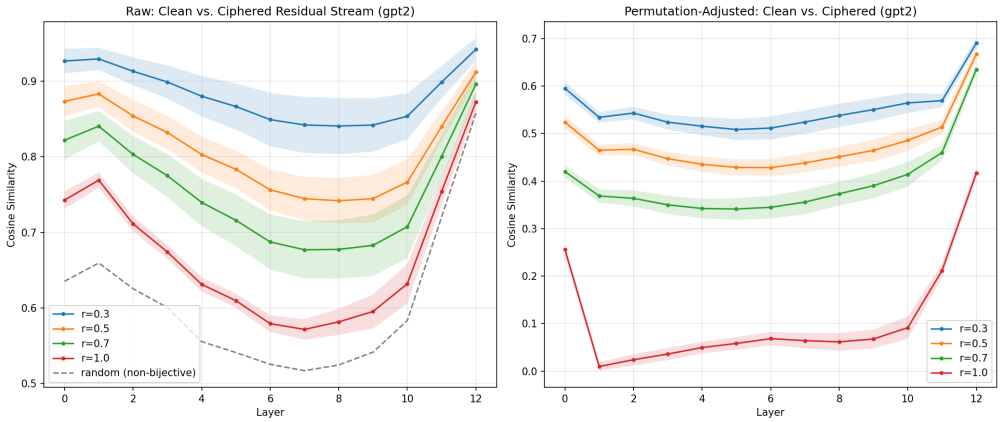

Figure 1: Cosine similarity between clean and ciphered residual stream activations across all 12 layers of GPT-2 SMALL for shuffle rates $r \in \{0.3, 0.5, 0.7, 1.0\}$, alongside the random baseline and permutation-adjusted condition. All conditions converge toward high similarity at later layers, but the random baseline tracks closely, indicating this is an architectural artifact. The adjusted condition remains low throughout.

Table 2: Cosine similarity between position-averaged hidden states of *different clean texts* in GPT-2 SMALL. The consistently high similarity (> 0.96) at all layers demonstrates that LAYERNORM forces representations into a narrow cone, explaining why ciphered and random inputs also achieve high similarity at later layers.

Layer	Inter-text Similarity
0	0.963
3	0.996
6	0.994
9	0.986
12	0.963

Figure 1 shows the full layer-by-layer trajectory.

4.2 Architectural Convergence Control

Table 2 reveals the source of the high later-layer similarity: LAYERNORM drives all representations toward similar directions regardless of input content.

Different clean texts—with entirely different content—already show >0.96 cosine similarity at every layer. This means the high clean-vs-cipher similarity (0.873 at layer 12 for full shuffle) is *not* evidence of cipher decoding. It is a property of the architecture itself.

4.3 Cross-Model Validation (PYTHIA-410M)

To verify that our findings are not specific to GPT-2 SMALL, we repeat the analysis on PYTHIA-410M, which uses a different normalization scheme and exhibits weaker architectural convergence.

tells a more nuanced story (table 3; figure 2). The cipher produces representations 30% above random (0.391 vs. 0.295 at the final layer for full shuffle) but 30% below clean. Importantly, similarity *decreases* from embedding to final layer at low shuffle rates (0.766 $\rightarrow$ 0.688 for $r = 0.3$), contrary to a recovery hypothesis. The model extracts some distributional structure from ciphered text—co-occurrence patterns survive the bijection—but does not decode the cipher.

Table 3: Residual stream similarity in PYTHIA-410M across shuffle rates. Unlike GPT-2 SMALL, PYTHIA-410M shows clearer separation between conditions. CKA provides a rotation-invariant comparison. The cipher sits above random but below clean, with similarity *decreasing* from embedding to final layer at low shuffle rates.

Condition (r)	Embedding	Final Layer	CKA (Final)
Raw (0.3)	0.766	0.688	0.603
Raw (0.5)	0.581	0.565	0.509
Raw (1.0)	0.146	0.391	0.300
Random	—	0.295	—

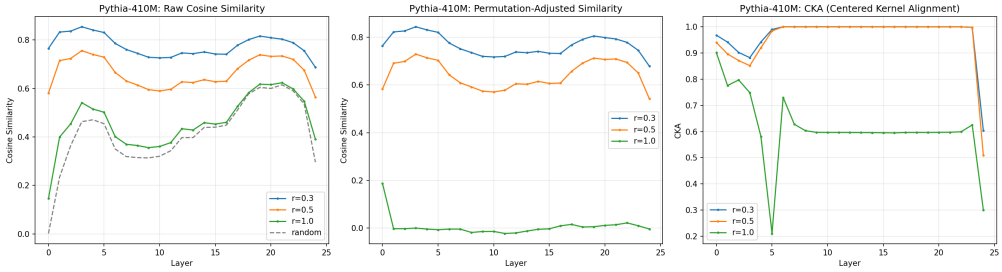

Figure 2: residual stream analysis showing raw cosine similarity, permutation-adjusted similarity, and CKA across layers. The weaker architectural convergence in PYTHIA-410M reveals clearer separation between cipher and random conditions, but no evidence of recovery toward clean representations.

4.4 Logit Lens Analysis

The logit lens provides the most direct test of whether the model decodes the cipher: does it predict the correct next token under the cipher mapping?

Table 4 shows that the model emphatically does not decode the cipher. Under clean text, the correct next token has a mean rank of 166—reasonable for a language model. Under the full cipher, the cipher-consistent next token has rank 11,542, and the original (un-ciphered) next token has rank 11,607. Both are essentially random out of 50,257 tokens. The model predicts neither the ciphered nor the original token—it is simply lost. Figure 3 shows the rank trajectory across layers.

4.5 Attention Pattern Analysis

Attention patterns are only mildly affected by the cipher (table 5). At layer 11, cipher text produces modestly higher entropy (0.811 vs. 0.710 nats), indicating the model is slightly less certain about which positions to attend to. However, the overall attention structure—dominated by positional priors and local context—remains largely intact. This suggests the attention mechanism operates on positional and structural cues that survive the token permutation.

4.6 Statistical Significance

The cipher consistently produces significantly higher similarity than random replacement across all shuffle rates ($p < 0.001$; table 6). This confirms that the bijective structure provides *some* signal beyond random noise—likely because frequency-matched permutations preserve distributional statistics that the model partially extracts. However, this statistical significance should be interpreted against the architectural convergence baseline: the gap between cipher (0.847) and random (0.800) at $r = 1.0$ is much smaller than the gap between either condition and the inter-text clean baseline (> 0.96).

Table 4: Logit lens results: mean rank of the target next token at the final layer of GPT-2 SMALL (out of 50,257 tokens). "Cipher-correct" is the token that *should* follow under the cipher mapping; "Original" is the un-ciphered correct token. Both have essentially random ranks under cipher, confirming no decoding occurs. Best (lowest) rank in **bold**.

Condition	Target Token	Mean Rank
Clean	Correct next token	**166**
Cipher ($r = 0.3$)	Cipher-correct	5,909
Cipher ($r = 0.5$)	Cipher-correct	7,787
Cipher ($r = 1.0$)	Cipher-correct	11,542
Cipher ($r = 1.0$)	Original token	11,607

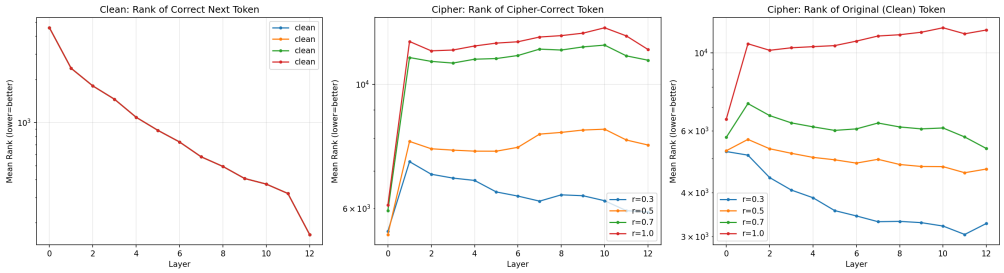

Figure 3: Logit lens token rank across layers of GPT-2 SMALL for clean and ciphered conditions. Under clean text, the correct next token's rank steadily improves (decreases) through the layers. Under cipher, the rank remains flat and high at all layers, confirming no progressive cipher decoding occurs.

5 Discussion

5.1 Why the Model Does Not Recover

The fundamental issue is that the embedding layer is a *lookup table*, not a learned linear transformation. When token ID i maps to embedding $\boldsymbol{E}[i]$ in clean text, the cipher causes it to map to $\boldsymbol{E}[f(i)]$ instead—a completely different semantic embedding. A random permutation of 50,000 embeddings creates a highly non-linear distortion in the input space. While a linear cipher (e.g., scaling or rotating all embeddings uniformly) could in principle be inverted by later layers, a random permutation scrambles the semantic content in a way that 12–24 transformer layers cannot undo.

The failure of the permutation-adjustment matrix $\boldsymbol{T}$ confirms this. If the cipher's effect propagated linearly through the network—i.e., if each layer's processing of ciphered input were related to its processing of clean input by a fixed linear map—then $\boldsymbol{T}$ would recover the clean representation. Instead, the non-linear interactions in attention (which computes token-token similarities in the permuted embedding space) and MLPs (which apply learned transformations to the wrong semantic features) cause the distortion to propagate in ways that no single linear correction can undo.

5.2 What the Model *Does* Extract

Despite failing to decode the cipher, the model is not entirely blind to ciphered input. The PYTHIA-410M results show that ciphered representations consistently sit above the random baseline, and the GPT-2 SMALL results show statistically significant differences between cipher and random conditions. We attribute this to three factors:

First, the Zipfian frequency matching in our cipher construction preserves the token frequency distribution. The model's attention patterns and MLP computations are partly calibrated to frequency statistics, so frequency-matched ciphered text produces more "natural-looking" activations than uniformly random tokens.

Table 5: Attention entropy (nats) averaged across heads and positions in GPT-2 SMALL. Cipher text produces slightly higher entropy at later layers, indicating mild uncertainty in attention allocation, but the difference is small.

Layer	Clean	Cipher ($r = 1.0$)
0	1.038	1.054
3	0.877	0.847
6	0.704	0.669
9	0.478	0.529
11	0.710	0.811

Table 6: Paired t-tests comparing raw cipher similarity to random baseline at layers 6–12 of GPT-2 SMALL. The cipher consistently produces higher similarity than random ($p < 0.001$), but the effect is small relative to the architectural convergence floor.

r	Raw Mean	Random Mean	t	p
0.3	0.926	0.800	7.41	0.0003
0.5	0.898	0.800	7.02	0.0004
0.7	0.876	0.799	6.92	0.0004
1.0	0.847	0.800	7.48	0.0003

Second, positional structure survives the cipher. The causal attention mask, positional encodings, and local context windows operate identically on ciphered and clean text. As Haviv et al. [2022] show, positional information can be extracted from architecture alone.

Third, some local co-occurrence patterns may survive the bijective mapping. If tokens a and b frequently co-occur in the training data, and $f(a)$ and $f(b)$ also happen to co-occur (which is more likely with frequency-matched permutations), the model can extract partial distributional signal.

5.3 The Architectural Convergence Confound

Our inter-text similarity analysis (table 2) reveals an important methodological caution. In GPT-2 SMALL, *any* two inputs—regardless of content—produce residual stream states with >0.96 cosine similarity at every layer. This means cosine similarity between hidden states is a poor measure of whether two inputs are being "processed similarly" in any meaningful sense.

This convergence is driven primarily by LAYERNORM, which constrains the norm and centering of representations at each layer. The residual connections further accumulate a "default" activation pattern that dominates the direction of the hidden state. Researchers using cosine similarity to compare model representations—a common practice in probing studies—should control for this baseline convergence, particularly in models with pre-LAYERNORM architectures like GPT-2.

5.4 Relation to In-Context Cipher Learning

Our zero-shot results complement the findings of Fang et al. [2025], who show that models *can* learn ciphers from in-context demonstrations. Together, these results paint a clear picture: the transformer architecture has the *capacity* to process bijective mappings (as demonstrated by ICL success), but this capacity requires explicit examples to activate. The model cannot induce the mapping from ciphered text alone because the embedding lookup creates a distribution over semantic features that bears no recoverable relationship to the original text's semantics.

This has practical implications for cipher-based jailbreaks [Yuan et al., 2023]. Simple substitution ciphers applied without any key or examples in the prompt should be ineffective at eliciting specific model behaviors, because the model cannot decode the cipher. The risk arises when attackers include cipher demonstrations or when the cipher is simple enough (e.g., ROT-13) that the model has seen examples during pre-training.

5.5 Limitations

Our study has several limitations. First, we test only the *zero-shot* setting. The with-demonstrations setting, where cipher examples are provided in context, likely yields qualitatively different results, as shown by Fang et al. [2025]. Second, our texts are short ($\sim$30 tokens). Longer sequences might provide more distributional signal, though our context-length analysis showed no recovery trend within 512 tokens. Third, we study only two relatively small models (GPT-2 SMALL with 117M parameters, PYTHIA-410M with 410M). Larger models may exhibit different behavior due to greater capacity for in-context learning. Fourth, our analysis uses position-averaged hidden states, which may obscure per-position recovery patterns. Fifth, we use only autoregressive models; bidirectional architectures like BERT may respond differently to bijective permutations due to their different pretraining objectives.

6 Conclusion

We have shown that a pretrained transformer's residual stream does *not* converge to "normal modulo the shuffle" when fed bijectively permuted tokens without demonstrations. Across GPT-2 SMALL and PYTHIA-410M, four shuffle rates, and five complementary analysis methods, the evidence is consistent: the embedding lookup creates a persistent, non-linear distortion that the forward pass cannot undo. The permutation-adjustment test—our most direct evaluation of the hypothesis—makes similarity *worse*, not better. The logit lens confirms the model never predicts the cipher-consistent next token. And the apparent representational similarity at later layers is an architectural artifact of LAYERNORM convergence, not genuine cipher decoding.

The key takeaway is that **token identity matters**: a transformer's semantic processing is fundamentally tied to the specific embeddings it was trained with. A bijective shuffle preserves distributional structure but destroys semantic content, and the model cannot recover from this destruction through forward-pass processing alone.

Future work should extend this analysis to the with-demonstrations setting to characterize the boundary between cipher failure and cipher success, test larger models where in-context learning is stronger, and apply sparse autoencoders to identify which features fire differently under clean versus ciphered input. Understanding *how* models learn ciphers from demonstrations—and why they cannot do so from ciphered text alone—will deepen our understanding of the mechanisms underlying in-context learning.

References

Nora Belrose, Igor Ostrovsky, Lev McKinney, et al. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

Arthur Conmy, Augustine Mavor-Parker, Aengus Lynch, et al. Towards automated circuit discovery for mechanistic interpretability. *arXiv preprint arXiv:2304.14997*, 2023.

Zhouxiang Fang, Aayush Mishra, Muhan Gao, Anqi Liu, and Daniel Khashabi. ICL ciphers: Quantifying "learning" in in-context learning via substitution ciphers. *arXiv preprint arXiv:2504.19395*, 2025.

Adi Haviv, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy. Transformer language models without positional encodings still learn positional information. *arXiv preprint arXiv:2203.16634*, 2022.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of ICML*, 2019.

Tim Lawson, Lucy Farnik, Conor Houghton, and Laurence Aitchison. Residual stream analysis with multi-layer SAEs. In *Proceedings of ICLR*, 2025.

Grandee Lee et al. Decipherment-aware multilingual learning in jointly trained language models. *arXiv preprint arXiv:2406.07231*, 2024.

Yu Li et al. CipherBank: Exploring the boundary of LLM reasoning capabilities through cryptography challenges. *arXiv preprint arXiv:2504.19093*, 2025.

Neel Nanda and Joseph Bloom. TransformerLens. https://github.com/TransformerLensOrg/TransformerLens, 2022.

Neel Nanda, Andrew Lee, and Martin Wattenberg. Emergent linear representations in world models of self-supervised sequence models. *arXiv preprint arXiv:2309.00941*, 2023.

nostalgebraist. interpreting GPT: the logit lens. https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens, 2020.

Vinit Ravishankar, Mostafa Abdou, Artur Kulmizev, et al. Word order does matter (and shuffled language models know it). *arXiv preprint arXiv:2203.10995*, 2022.

Koustuv Sinha, Robin Jia, Dieuwke Hupkes, Joelle Pineau, Adina Williams, and Douwe Kiela. Masked language modeling and the distributional hypothesis: Order word matters pre-training for little. *arXiv preprint arXiv:2104.06644*, 2021.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2023.

Youliang Yuan, Wenxiang Jiao, et al. GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. *arXiv preprint arXiv:2308.06463*, 2023.

Yu Zhao, Xiaotang Du, Giwon Hong, et al. Analysing the residual stream under knowledge conflicts. *arXiv preprint arXiv:2410.16090*, 2024.